

Entities and Groups

Every DCGM operation begins with a target. The host engine represents a target as a typed *entity* and represents reusable sets of targets as *entity groups*. Discover entities from the host engine that will perform the operation; then select only the objects that the operation and the administrator intend to affect.

For exact discovery options and entity-selector syntax, see [dcgmi discovery](#), [dcgmi group](#), and [Entity selectors](#).

The Entity Namespace

An entity is identified by both its family and its numeric ID. `gpu:0` and `cpu:0` are different entities. A bare number is accepted as a GPU ID by some commands for compatibility, but an explicit prefix makes automation and mixed-entity groups unambiguous.

DCGM 4.6 entity families

Entity	Selector	Inventory source
Physical GPU	<code>gpu:<id></code>	<code>dcgmi discovery --list</code> ; add <code>--all</code> for inactive GPUs.
MIG GPU instance	<code>instance:<id></code>	<code>dcgmi discovery --compute-hierarchy</code> .
MIG compute instance	<code>compute_instance:<id></code>	<code>dcgmi discovery --compute-hierarchy</code> .
Virtual GPU	<code>vgpu:<id></code>	The vGPU management or API inventory. DCGM 4.6 has no dedicated <code>dcgmi discovery</code> operation for vGPU IDs.
NVSwitch	<code>nvswitch:<id></code>	<code>dcgmi discovery --list</code> .
NVLink port	<code>gpu_link:<gpu>:<port></code> , <code>switch_link:<switch>:<port></code> , or <code>link:<packed-id></code>	<code>dcgmi nvlink --link-status --show-entity-ids</code> .
NVIDIA CPU	<code>cpu:<id></code>	<code>dcgmi discovery --list</code> .
NVIDIA CPU core	<code>core:<id></code>	CPU ranges in <code>discovery --list</code> or <code>discovery --info a --cpuid <id></code> .
ConnectX device	<code>cx:<id></code>	<code>dcgmi discovery --list</code> .

The host engine reports only the entities supported by the installed DCGM release, loaded feature modules, platform software, and detected hardware. It is normal for most hosts to contain only a subset of these families.

Entity family also determines which fields and operations make sense. A GPU temperature field cannot be assumed to work for an NVLink port, and a command that accepts a general entity selector can still restrict its operation to particular families. Validate both the selector grammar and the consuming command.

Build the Runtime Inventory

Start with the same host engine that later commands will use:

```
$ dcgmi discovery --list
```

This lists active GPUs and the NVSwitches, ConnectX devices, and supported NVIDIA CPUs known to that engine. Include detached or otherwise inactive GPUs when investigating missing hardware:

```
$ dcgmi discovery --list --all
```

Inactive GPU output includes a status such as `DETACHED`, `LOST`, or `INACCESSIBLE`. Identity values marked `last known` came from the host engine's earlier view; they do not prove that the device is currently reachable.

MIG instances form a parent-child hierarchy rather than a flat GPU list:

```
$ dcgmi discovery --compute-hierarchy
```

Read the hierarchy after MIG configuration changes. A GPU-instance or compute-instance ID is meaningful with its entity family and parent placement; it is not a physical GPU ID.

NVLink ports are enumerated with link status rather than the general discovery list:

```
$ dcgmi nvlink --link-status --show-entity-ids
```

The structured `gpu_link` and `switch_link` forms express the parent device and port directly. The packed `link` ID is useful when software consumes the enumeration output. Continue with [Topology and NVLink](#) before interpreting a link state or counter.

Correlate IDs with durable identity

DCGM IDs are convenient indices in one host engine's runtime namespace. Do not use a numeric GPU ID as a durable asset identifier or assume that IDs from two hosts refer to equivalent hardware.

Query the selected GPU's identity:

```
$ dcgmi discovery --info a --gpuid 0
```

Use the GPU UUID when correlating with a scheduler, inventory database, or long-lived monitoring series. The PCI bus ID is useful for correlating with the host's PCI topology. Names and serial numbers can add context but should not replace a unique identity.

Other discovery attribute sets answer configuration questions:

- `p` reports power-limit values;
- `t` reports thermal limits;
- `c` reports supported application-clock pairs; and
- `w` reports workload power profiles.

For example, query identity, power, and thermal attributes before selecting a configuration value:

```
$ dcgmi discovery --info apt --gpuid 0
```

Discovery reports capability and current inventory. It does not reserve an entity, bind a workload, or test the device under load.

Groups Are Reusable Targets

A DCGM entity group is a named set of entity IDs stored by the host engine. Commands and APIs can refer to its group ID instead of repeating the same entity list. Groups can overlap, and a group can contain more than one entity family when the consuming operation supports those families.

Groups are selectors only. Creating a group:

- does not allocate or isolate hardware;
- does not change `CUDA_VISIBLE_DEVICES` or device-cgroup access;
- does not start a field watch;
- does not apply configuration; and
- does not imply that every DCGM operation accepts its members.

The scheduler, container runtime, and operating system remain responsible for placement and access control.

Choose a group lifetime

Use built-in groups for simple system-wide operations. Commands that accept them use names such as `all_gpus` and `all_nvswitches`. List the current groups instead of hard-coding their numeric IDs:

```
$ dcgmi group --list
```

Create a long-lived user group when a management integration repeatedly targets the same site-defined set. Create a transient group when its membership mirrors a scheduler allocation. A group created by `dcgmi` remains after that client exits, but it exists only in the running host engine and is lost when the engine exits.

Deleting a group removes the selector. It does not undo settings already applied to its members, delete telemetry retained for unrelated watches, or release the resources from a scheduler.

Example: mirror an allocation

Suppose the scheduler assigned GPUs with UUIDs that current discovery maps to DCGM GPU IDs 0 and 2. Create and populate the group in one command:

```
$ dcgmi group --create batch-4815 --add gpu:0,gpu:2
```

The host engine prints the new numeric group ID. Inspect membership before using the group:

```
$ dcgmi group --group <group-id> --info
```

The group can now be used by compatible monitoring, topology, health, configuration, diagnostic, or statistics operations. During epilogue cleanup:

```
$ dcgmi group --delete <group-id>
```

For a node-wide custom group, ask the host engine to populate every available GPU:

```
$ dcgmi group --create managed-gpus --default
```

Use a mixed group only for an operation documented to accept it:

```
$ dcgmi group --create platform-entities \  
  --add gpu:0,nvswitch:0,cx:0
```

Operational Failure Checks

When a target cannot be selected or an operation returns no data, separate the possible boundaries:

1. Run discovery against the intended host engine and confirm the entity family and ID.
2. For a GPU, compare its UUID and active status rather than relying on its old numeric ID.
3. Inspect the group and confirm that the entity is a member.
4. Check that the command supports every family in the group.
5. For MIG, re-read the compute hierarchy after reconfiguration.
6. For NVSwitch, CPU, ConnectX, or vGPU entities, check the corresponding feature module and platform prerequisite.
7. If a GPU is inactive, continue with [Detached GPUs](#) rather than treating cached identity as live telemetry.

Further Reading

- [Fields and Watches](#) combines entity selections with field selections.
- [Topology and NVLink](#) explains relationships among discovered GPUs and ports.
- [dcqmi discovery](#) is the exhaustive inventory and attribute reference.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.